Advanced Predictive Digital Twin for Industrial IoT Monitoring Using ONNX Runtime

Presented by

Bhavya Keerthi K
22BCE8819

Project completed under internship at
Wiman Communication Technologies

School of Computer Science and Engineering
VIT-AP University, Amaravati, India

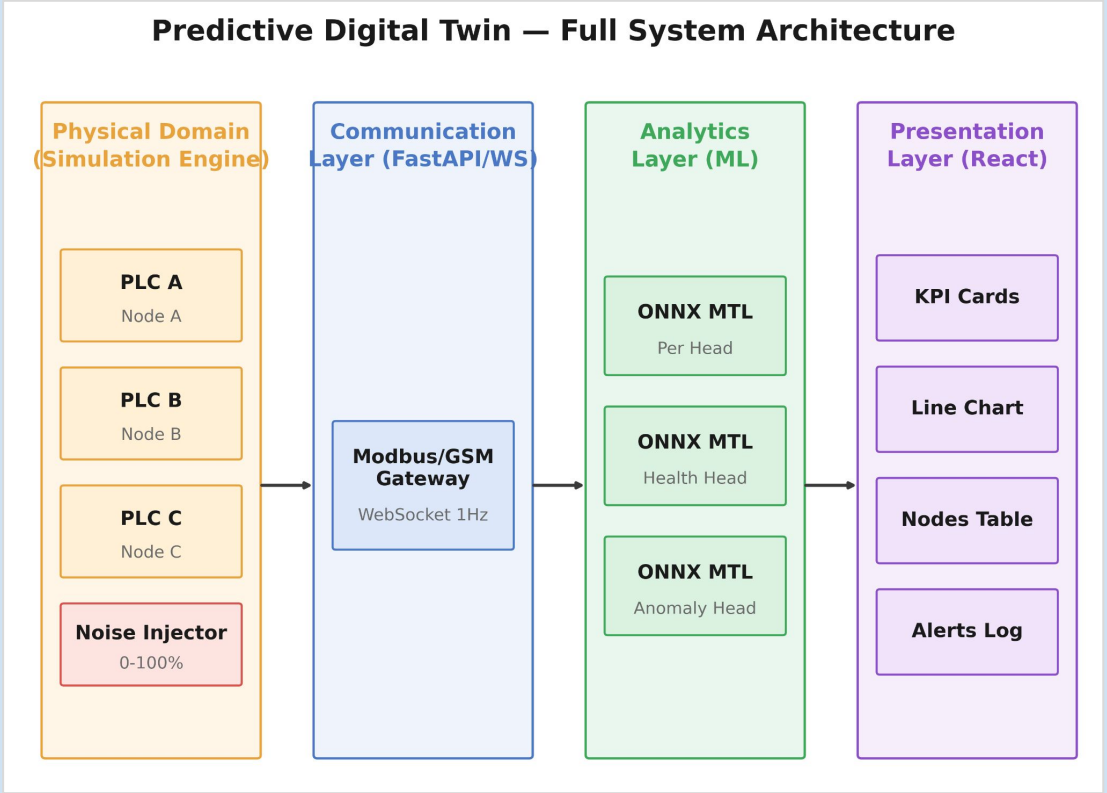
Presentation Outline

VIT-AP UNIVERSITY

- Introduction
  - Project Overview
- Project Objectives
- Project Methodology
  - System Architecture
  - Key Components/Modules
  - Algorithms Used/Technologies, Frameworks, and Tools Used
- Key Results
  - Specific measurable outputs
- Summary (Findings, Limitations, Future Directions)
- References

Abstract & System Overview

- Predictive digital twin for a Modbus-over-GSM industrial network
- Simulated PLC nodes stream telemetry via a FastAPI backend over WebSocket
- An ONNX multi-task learning model performs real-time inference on the stream
- A React dashboard presents live operational status to the operator
- Key result: ONNX inference is ~11.4x faster than PyTorch (11.00 ms → 0.96 ms), accuracy preserved within 1×10^{-4}



Objectives & Scope

Project Objectives

- Simulate **real-time multi-node PLC telemetry** in an industrial environment.
- Stream live telemetry data to a web dashboard using **WebSocket** communication.
- Perform **real-time predictive health and risk estimation** using an **ONNX-based Multi-Task Learning model**.
- Provide an **operator-friendly dashboard** with KPI cards, charts, and alert notifications.
- Implement **closed-loop polling-rate optimization** for adaptive monitoring based on predicted system risk.

Project Scope

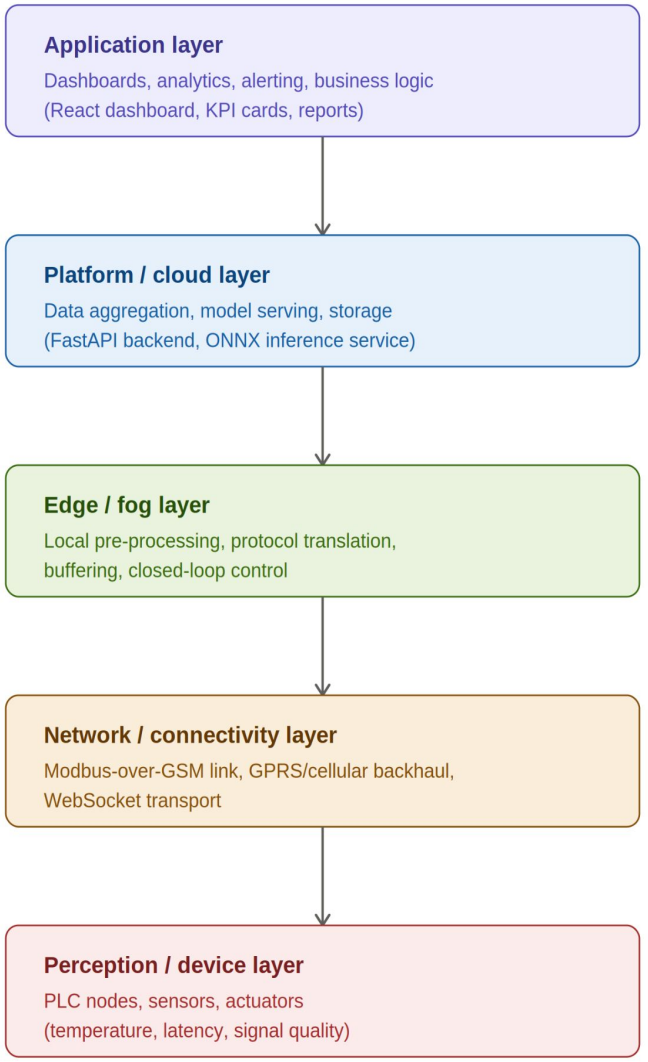
- Develop and validate a **complete software pipeline** for Industrial IoT monitoring over a **simulated Modbus-over-GSM network**.
- **No physical PLC hardware** is used; all telemetry, communication, and network behavior are simulated for experimentation and evaluation.

Layered System Architecture

VIT-AP UNIVERSITY

Architecture Layers

- **Perception Layer:** Collects telemetry data from PLCs, sensors, and actuators.
- **Network Layer:** Transfers data using **Modbus-over-GSM** and WebSocket communication.
- **Edge/Fog Layer:** Performs local preprocessing, protocol translation, buffering, and closed-loop control.
- **Platform/Cloud Layer:** Handles data aggregation, ONNX model inference, storage, and backend services.
- **Application Layer:** Provides dashboards, KPI visualization, alerts, reports, and maintenance insights.



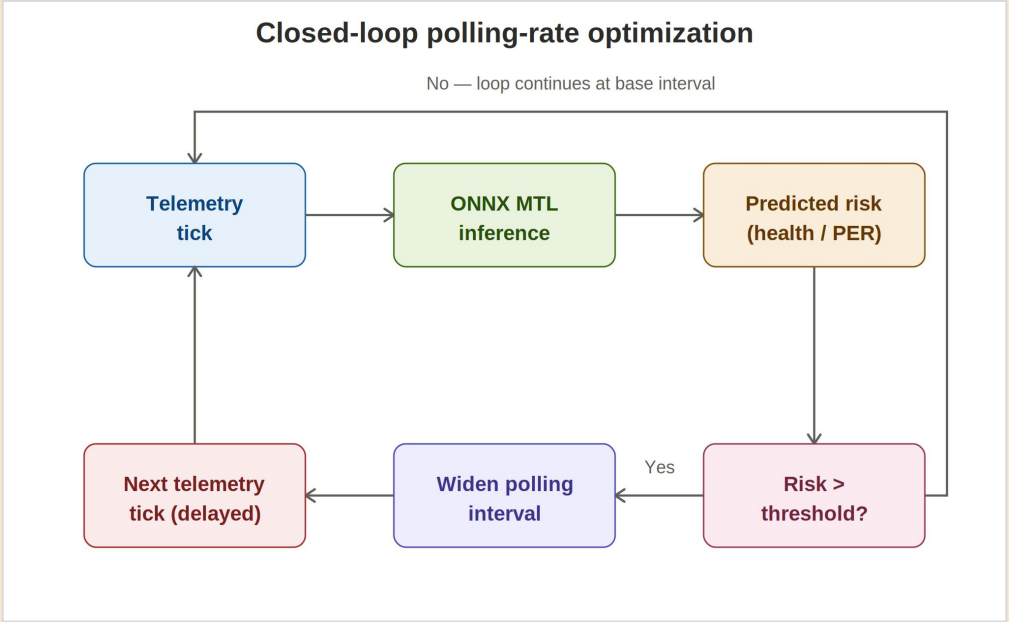
Data and control flow between layers (telemetry flows up; control/config flows down)

Closed-Loop Polling-Rate Optimization

VIT-AP UNIVERSITY

Workflow

- **Telemetry Tick:** Sensor data is collected periodically.
- **ONNX MTL Inference:** The model predicts **equipment health** and **Packet Error Rate (PER)**.
- **Risk Assessment:** The predicted risk is compared with a predefined threshold.
- **Adaptive Polling:**
  - **High Risk:** Maintain the normal polling interval for continuous monitoring.
  - **Low Risk:** Increase (widen) the polling interval to reduce communication overhead.
- The updated interval is used for the **next telemetry cycle**, forming a closed-loop feedback system.

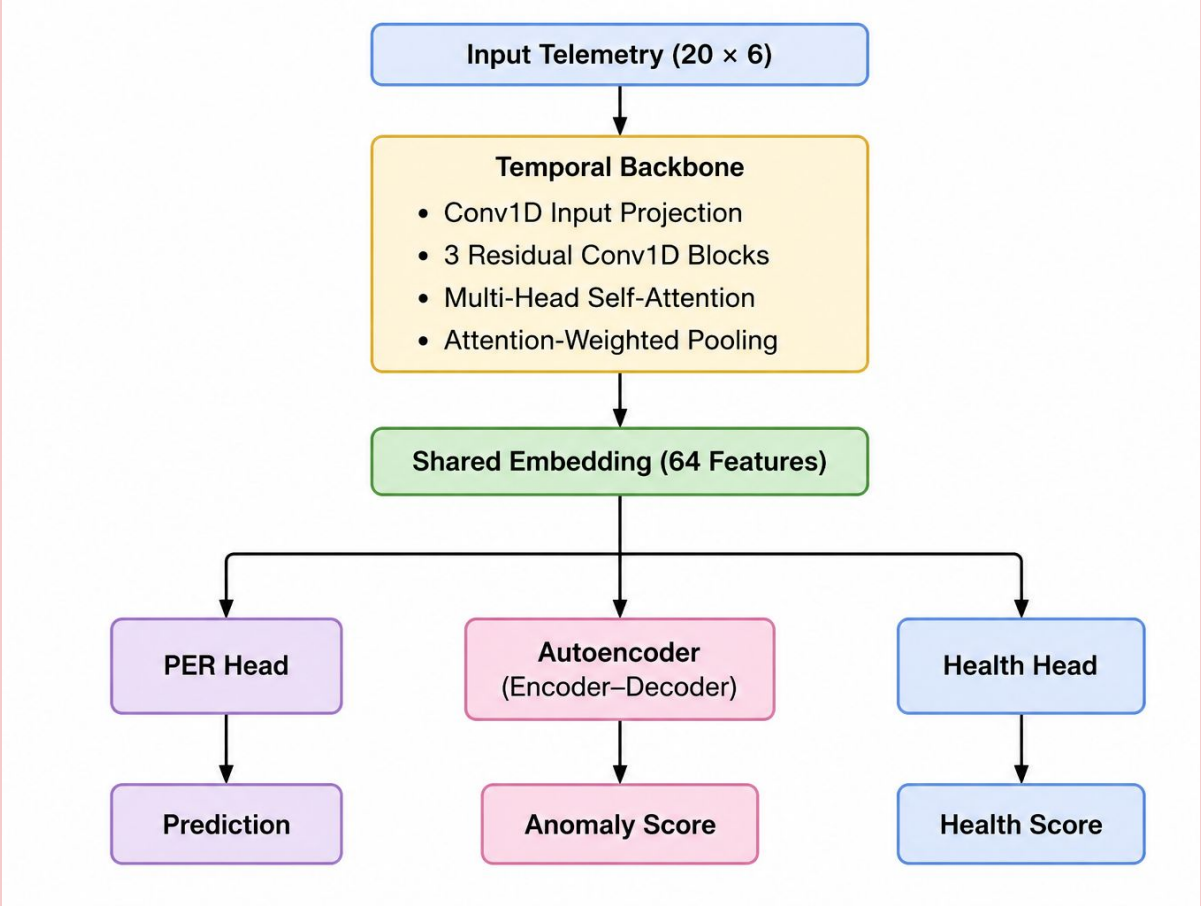


Multi-Task Learning Model Architecture

VIT-AP UNIVERSITY

Key Features

- **Shared Backbone:** Learns common temporal patterns from sensor data.
- **PER Head:** Generates the primary prediction (e.g., Remaining Useful Life or regression output).
- **Autoencoder:** Computes reconstruction error for anomaly detection.
- **Health Head:** Predicts equipment health (0–100%) and flags failures when **Health Score < 40**.
- **Attention Mechanism:** Identifies the most important time steps influencing the prediction.

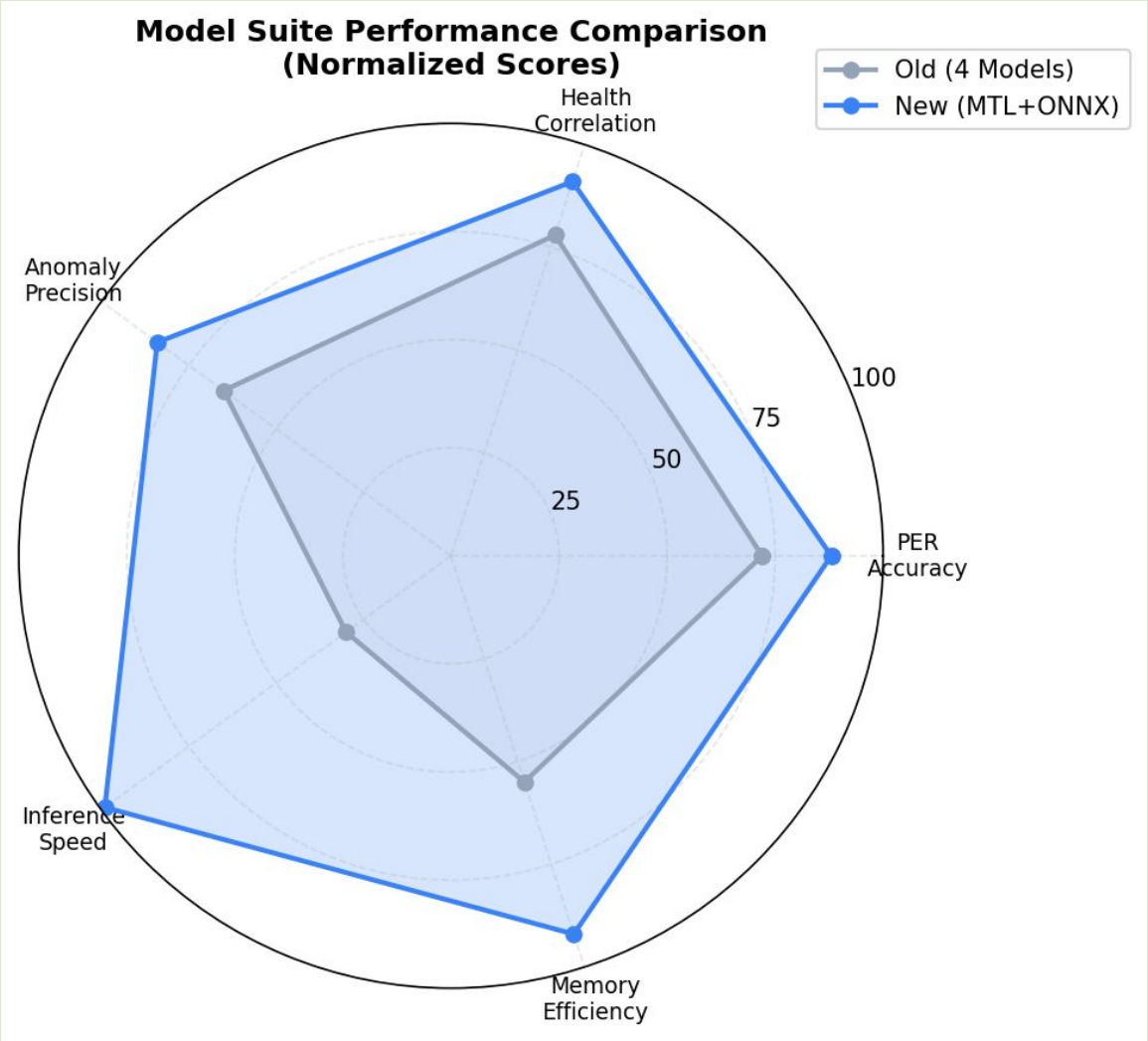


Model Performance Comparison (Radar Chart)

VIT-AP UNIVERSITY

Evaluation Metrics

- **PER Accuracy:** Accuracy of the primary prediction task.
- **Health Correlation:** Agreement between predicted and actual equipment health.
- **Anomaly Precision:** Accuracy of anomaly detection.
- **Inference Speed:** Time required to generate predictions.
- **Memory Efficiency:** Resource utilization during deployment.



Simulator & Telemetry Pipeline

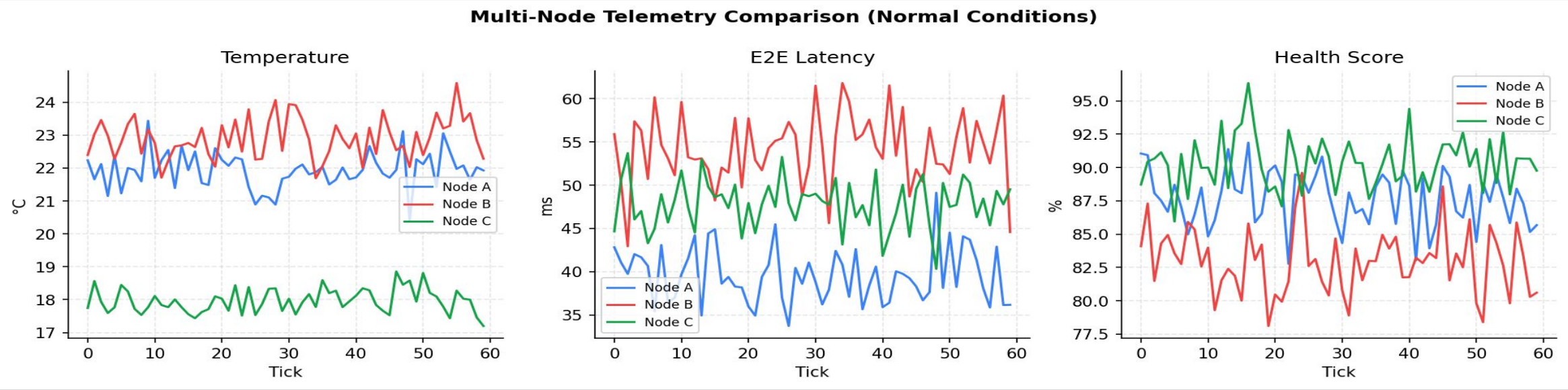
VIT-AP UNIVERSITY

Objective

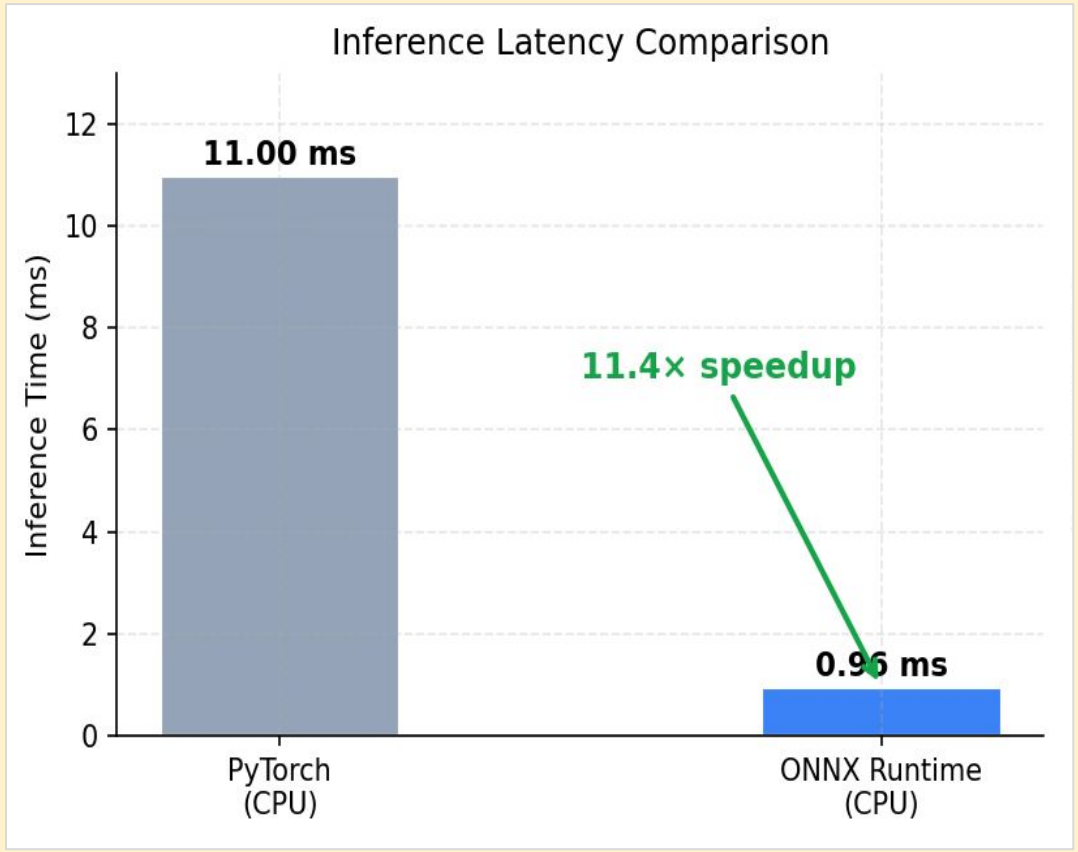
Compare the real-time performance of three industrial nodes under normal operating conditions.

Key Observations

- **Temperature:** All nodes maintain stable temperatures with minor fluctuations.
- **E2E Latency:** Communication latency remains within acceptable limits for all nodes.
- **Health Score:** All nodes achieve high health scores (**80–95%**), indicating healthy operation.
- **Node C** shows the highest health score, while **Node B** experiences slightly higher latency and temperature.



ONNX vs. PyTorch Performance



- **PyTorch Inference Time: 11.00 ms**
- **ONNX Runtime Inference Time: 0.96 ms**
- **Performance Gain: 11.4× faster** with ONNX Runtime.
- Both models satisfy the **1 Hz (1000 ms)** real-time requirement, with ONNX using significantly less processing time.
- **Output Validation:** All five model outputs matched the PyTorch model within an **absolute tolerance of 1×10^{-4}**, confirming that prediction accuracy is preserved after ONNX conversion.
- **ONNX Runtime provides much faster inference while maintaining the same prediction accuracy, making it suitable for real-time Digital Twin deployment.**

Network Noise Injection Analysis

VIT-AP UNIVERSITY

Observations

- **Network Noise:** Noise increases from **10% to 60%** between **40–80 ticks**, simulating a high-noise period.
- **Temperature:** All nodes show a slight increase but remain below the warning (30°C) and critical (35°C) thresholds.
- **E2E Latency:** Increases from approximately **65 ms** to **190 ms** during the high-noise interval and returns to normal afterward.
- **Packet Error Rate (PER):** Both the **predicted PER** and **actual packet loss** increase to around **40–45%**, exceeding the **15% ML trigger threshold**.

❑ The proposed Digital Twin accurately detects network degradation during high GSM noise conditions. The close agreement between predicted and actual PER demonstrates the effectiveness of the MTL model for **real-time network monitoring and predictive maintenance**.

MTL Training Convergence

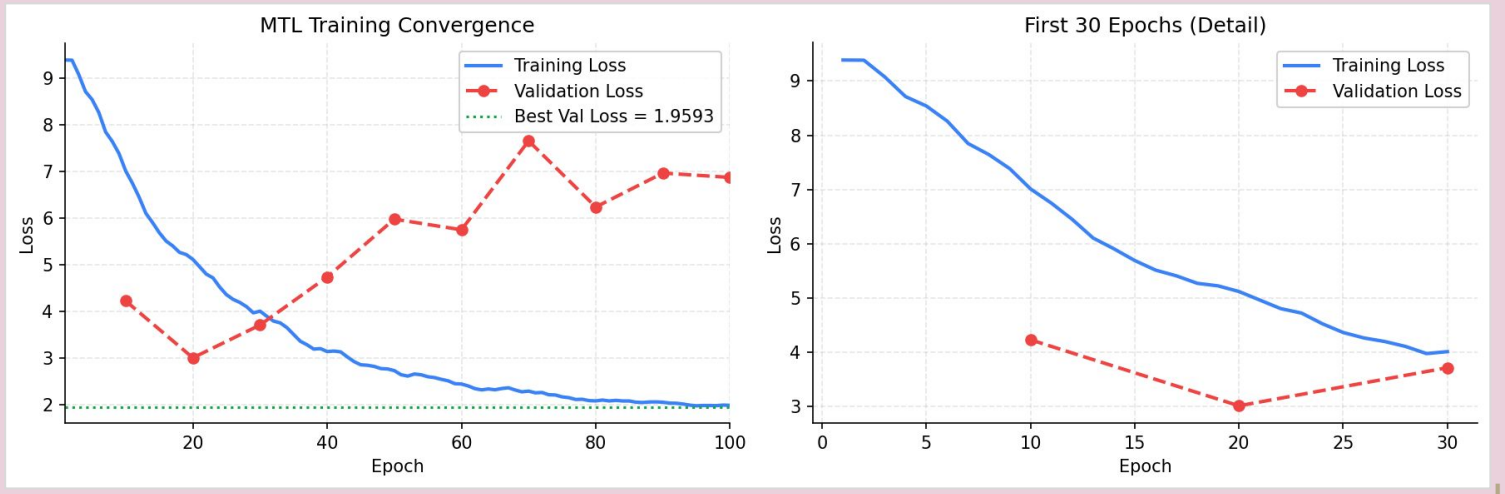
VIT-AP UNIVERSITY

Observations

- **Training loss** decreases steadily from **9.4** to **1.96**, indicating successful model learning.
- **Validation loss** reaches its minimum (**3.0**) around **Epoch 20**, then fluctuates in later epochs.
- The first **30 epochs** show rapid improvement, with both training and validation losses decreasing significantly.
- **Best training loss achieved: 1.9593**.

Conclusion

The MTL model converges successfully during training, learning meaningful representations within the first few epochs. The stabilization of training loss demonstrates effective optimization, while the validation trend indicates the model achieves good generalization before slight overfitting in later epochs.



Conclusion & Future Work

VIT-AP UNIVERSITY

Conclusion

- Developed a **Unified Multi-Task Learning Digital Twin** for predictive maintenance over a **Modbus-over-GSM** industrial network.
- Successfully integrated **multi-task learning** to simultaneously predict equipment health, anomalies, and Packet Error Rate (PER) using a shared temporal backbone.
- Exported the trained model to **ONNX Runtime**, achieving **11.4× faster inference** (11.00 ms → 0.96 ms) while maintaining prediction accuracy within $\mathbf{1 \times 10^{-4}}$.
- Experimental results demonstrate improved **real-time performance, memory efficiency, and reliable health monitoring** for industrial systems.
- The proposed framework is suitable for **edge deployment** and supports efficient real-time predictive maintenance in Industrial IoT environments.

Future Work

- Validate the framework using **real industrial Modbus-over-GSM deployments** instead of simulated datasets.
- Extend support to **4G/5G and Industrial IoT communication networks** for improved reliability and scalability.
- Integrate **adaptive online learning** so the model can continuously learn from new telemetry data.
- Deploy the ONNX model on **edge AI devices** (e.g., NVIDIA Jetson or Raspberry Pi) for low-latency industrial applications.
- Develop an **interactive dashboard** with real-time alerts, fault visualization, and maintenance recommendations for operators.

VIT-AP UNIVERSITY

THANK YOU!
ANY QUESTIONS?